

Coding Challenge 03

Three integrated programming challenges that apply control statements, Boolean logic, compound conditions, nested decisions, input validation, decision tables, mathematical formulas, and systematic testing.

Control Statements

Decision Logic

Input Validation

Business + Mathematics

Console-Based Python

Engr. Bob Mathew D. Sunga, MSc.

Challenge Objective

This coding challenge asks you to move beyond isolated `if` statements. Each problem contains several layers of reasoning. You must first understand the rules, validate the data, determine the correct conditions, calculate intermediate values, choose the appropriate decision structure, test multiple paths, and present the result as a clear analytical report.

The three problems intentionally use different domains so that you learn to recognize **decision logic as a transferable programming skill**, not as syntax tied to only one kind of problem.

Core Concepts

- ✓ Boolean expressions
- ✓ Comparison and logical operators
- ✓ `if`, `if-else`, `if-elif-else`
- ✓ Compound conditions

✓ Nested decisions

✓ Input validation

✓ Decision tables

✓ Boundary testing

Important: A program that runs without an error is not automatically correct. Your program must implement the stated rules accurately, handle boundary cases, and produce the correct result for different input combinations.

BEFORE YOU START

General Programming Requirements

All three programs should demonstrate the Python decision-making skills developed in the previous lessons. The emphasis is on correctness, readable logic, and the ability to explain how the program reaches its final result.

Required Programming Practices

- Use meaningful variable names.
- Convert user input to appropriate numeric types.
- Validate inputs before performing calculations that depend on them.
- Use the appropriate control structure for each decision.
- Use comments to identify major sections of the program.
- Clearly label all displayed results.
- Format decimal values consistently.

Scope

- No graphical interface is required.
- No NumPy, Pandas, or external mathematical library is required.
- No loops are required for the core solution.
- No classes are required.
- Custom functions are not required.
- Use the formulas and rules provided in this challenge.

Recommended program structure: Input → Validation → Intermediate Calculations → Classification / Decision Logic → Final Result → Verification / Report.

User Input

Validate

Calculate

Evaluate Rules

Decide

Report

1

Rule-Based Loan Decision Engine

Finance / Business Decision Logic

Build a console-based decision engine that evaluates a hypothetical loan application using applicant information, calculated debt burden, classification rules, and a final decision table.

Academic-use note: The thresholds below are instructional rules created for this programming challenge. They are not real lending standards and should not be treated as financial advice or actual credit policy.

Required Inputs

Input	Recommended Type	Description
Monthly Income	float	Applicant's gross monthly income.
Monthly Debt	float	Total existing monthly debt obligations.
Credit Score	int or float	Instructional credit score from 300 to 850.
Employment Duration	int	Number of months employed.
Requested Loan	float	Amount the applicant is requesting.
Existing Customer	str	Use yes or no. This should be normalized for comparison.

Input Validation Rules

- ✓ Monthly income must be greater than 0.
- ✓ Monthly debt cannot be negative.
- ✓ Credit score must be from 300 to 850 inclusive.
- ✓ Employment duration cannot be negative.
- ✓ Requested loan amount must be greater than 0.
- ✓ Existing customer status must be either yes or no.

If an input is invalid: display a clear validation message and do not present an approval decision based on invalid data.

Debt-to-Income Ratio

The program must calculate the percentage of monthly income already committed to monthly debt obligations.

$$DTI = \frac{\text{Monthly Debt}}{\text{Monthly Income}} \times 100$$

Debt-to-Income Ratio expressed as a percentage

Example: if monthly debt is ₦15,000 and monthly income is ₦60,000:

$$DTI = \frac{15000}{60000} \times 100 = 25\%$$

Credit Score Classification

Credit Score	Classification
750–850	Excellent
700–749	Good
650–699	Fair
300–649	High Risk

DTI Classification

DTI	Classification
30% or below	Low
Above 30% up to 40%	Moderate
Above 40% up to 50%	High
Above 50%	Very High

Maximum Recommended Loan

For this challenge only, calculate an instructional recommended limit using twelve months of income.

$$\text{Maximum Recommended Loan} = \text{Monthly Income} \times 12$$

Core Eligibility Decision Table

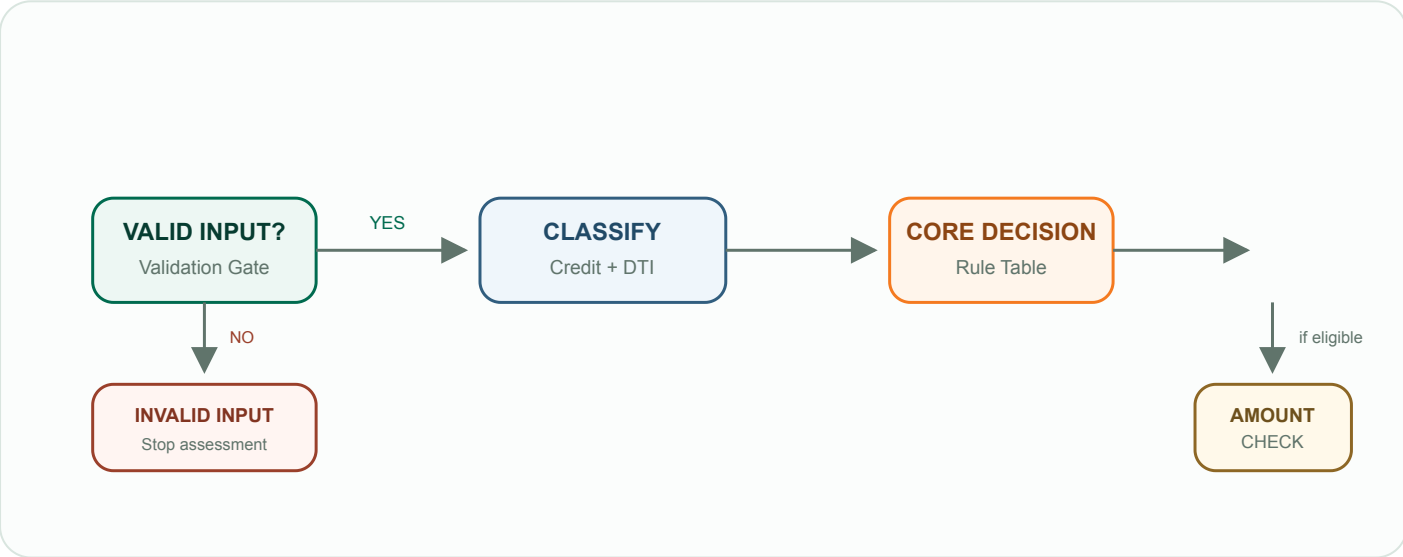
Use the following table as the primary business rule model. Rules should be interpreted from top to bottom where a more restrictive rule applies.

Credit Classification	DTI	Employment	Decision
High Risk	Any	Any	DECLINED
Any	Above 50%	Any	DECLINED
Any	Any	Below 6 months	DECLINED

Credit Classification	DTI	Employment	Decision
Excellent or Good	40% or below	12 months or more	APPROVED
Excellent or Good	50% or below	6 months or more	FOR REVIEW
Fair	40% or below	12 months or more	FOR REVIEW
All other valid combinations	—	—	DECLINED

Requested Amount Assessment

After the core eligibility decision, evaluate the requested amount. If an otherwise approvable request exceeds the maximum recommended loan, the final result should not remain automatically approved. It should be moved to **FOR REVIEW**.



Required Final Report

- ✓ Echo the applicant inputs.
- ✓ Display DTI and its classification.
- ✓ Display credit classification.
- ✓ Display maximum recommended loan.
- ✓ State whether the requested amount is within the instructional limit.
- ✓ Display the final decision: APPROVED, FOR REVIEW, DECLINED, or INVALID INPUT.
- ✓ Display a short reason explaining the decision.

LOAN DECISION ASSESSMENT

Monthly Income: PHP 60,000.00
Monthly Debt: PHP 15,000.00
Requested Loan: PHP 500,000.00
Employment Duration: 36 months
Credit Score: 735
Existing Customer: Yes

ASSESSMENT

Credit Classification: Good
Debt-to-Income Ratio: 25.00%
DTI Classification: Low
Maximum Recommended Loan: PHP 720,000.00
Requested Amount Status: Within Recommended Limit

FINAL DECISION

APPROVED

Reason: Credit, DTI, employment duration, and
requested amount satisfy the challenge rules.

Required Test Scenarios

Approved case: Good or Excellent credit, acceptable DTI, sufficient employment, and requested amount within the recommended limit.

Review case: A valid combination that matches a FOR REVIEW rule.

High-DTI decline: DTI above 50%.

Credit-risk decline: High Risk credit classification.

Invalid-input case: At least one invalid entry.

Boundary case: Test an exact boundary such as DTI = 40%, DTI = 50%, credit score = 700, or employment = 6 months.

Implementation Tips

- Validate income before dividing by it.
- Arrange credit and DTI classifications from the most specific/highest range downward.
- Implement restrictive decline rules before permissive approval rules.
- Keep the core eligibility decision separate from the requested-amount adjustment so the logic is easier to explain.
- Use parentheses in compound conditions when they improve readability.

2

Quadratic Equation and Parabola Analyzer

Mathematics / Algebra / Analytic Geometry

Create a program that accepts the coefficients of a quadratic equation, validates that the equation is truly quadratic, analyzes the discriminant, determines the roots when real roots exist, and reports important geometric properties of the corresponding parabola.

General Form

$$ax^2 + bx + c = 0$$

The program asks the user for the coefficients **a**, **b**, and **c**.

Validation Rule

A quadratic equation requires:

$$a \neq 0$$

If $a = 0$, the equation is not quadratic.

If $a = 0$: report that the equation is not quadratic and do not apply quadratic formulas that divide by $2a$.

Discriminant

The discriminant determines the nature of the roots.

$$D = b^2 - 4ac$$

Condition	Root Classification
$D > 0$	Two distinct real roots
$D = 0$	One repeated real root
$D < 0$	No real roots

Quadratic Formula

When the discriminant is positive, calculate two real roots:

$$x_1 = \frac{-b + \sqrt{D}}{2a}$$

$$x_2 = \frac{-b - \sqrt{D}}{2a}$$

When the discriminant is zero, the repeated root is:

$$x = \frac{-b}{2a}$$

For $D < 0$: this challenge only requires the message *No real roots*. Complex-number calculation is outside the required scope.

Axis of Symmetry

$$x = \frac{-b}{2a}$$

Vertex

First calculate the x-coordinate of the vertex:

$$x_v = \frac{-b}{2a}$$

Then substitute that value into the quadratic function:

$$y_v = ax_v^2 + bx_v + c$$

$$\text{Vertex} = (x_v, y_v)$$

Opening Direction and Vertex Type

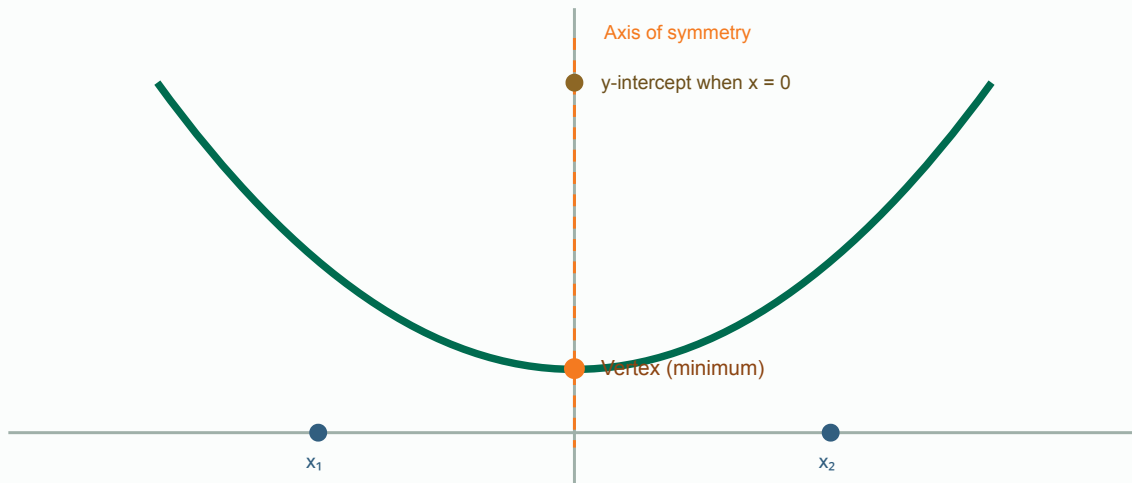
Condition	Parabola Direction	Vertex Type
$a > 0$	Opens upward	Minimum
$a < 0$	Opens downward	Maximum

Y-Intercept

When $x = 0$, the quadratic function becomes $y = c$. Therefore:

$$\text{Y-Intercept} = (0, c)$$

Visual Interpretation



Required Final Report

✓ Display the equation using the entered coefficients.

✓ Display the discriminant.

✓ State the root classification.

✓ Display real root values when they exist.

✓ Display the opening direction.

✓ Display the axis of symmetry.

✓ Display the vertex.

✓ State whether the vertex is a minimum or maximum.

✓ Display the y-intercept.

QUADRATIC EQUATION ANALYZER

Equation:

$$2.00x^2 - 8.00x + 6.00 = 0$$

DISCRIMINANT ANALYSIS

Discriminant: 16.00

Root Classification: Two Distinct Real Roots

Root 1: 3.00

Root 2: 1.00

PARABOLA PROPERTIES

Direction: Opens Upward

Axis of Symmetry: $x = 2.00$

Vertex: (2.00, -2.00)

Vertex Type: Minimum

Y-Intercept: (0.00, 6.00)

Required Test Scenarios

Two real roots: Choose coefficients that produce $D > 0$.

Repeated root: Choose coefficients that produce $D = 0$.

No real roots: Choose coefficients that produce $D < 0$.

Upward parabola: Use $a > 0$.

Downward parabola: Use $a < 0$.

Invalid quadratic: Use $a = 0$.

Implementation Tips

- Validate a before calculating any quantity that divides by $2a$.
- Calculate the discriminant once, store it in a variable, and reuse it.
- Only calculate the square root of D when D is non-negative.
- The axis of symmetry and vertex x-coordinate use the same formula, so calculate that quantity once and reuse it.
- For the required $D = 0$ test, use integer coefficients that produce an exact zero discriminant.

3

Two-Variable Linear System Analyzer

Mathematics / Linear Algebra / Analytic Geometry

Build a program that accepts two linear equations in two variables, validates them, uses determinants to classify the system, solves the system when a unique solution exists, and explains the geometric meaning of the result.

General Form of the System

$$a_1 x + b_1 y = c_1$$

$$a_2 x + b_2 y = c_2$$

The user enters six coefficients: $a_1, b_1, c_1, a_2, b_2, c_2$.

Equation Validation

Each equation must contain at least one variable coefficient. For this challenge, reject an equation when both variable coefficients are zero:

$$a_i = 0 \text{ and } b_i = 0$$

Such an entry does not represent a valid linear equation in x and y for this challenge.

Cramer's Rule Determinants

Calculate three determinants.

$$D = a_1 b_2 - a_2 b_1$$

$$D_x = c_1 b_2 - c_2 b_1$$

$$D_y = a_1 c_2 - a_2 c_1$$

System Classification

Condition	Classification	Number of Solutions	Geometry
$D \neq 0$	Consistent and Independent	One unique solution	Two lines intersect at one point
$D = 0, D_x = 0, D_y = 0$	Consistent and Dependent	Infinitely many solutions	The equations describe the same line
$D = 0$ and either $D_x \neq 0$ or $D_y \neq 0$	Inconsistent	No solution	The lines are parallel and distinct

Unique Solution Formula

Only when $D \neq 0$, calculate:

$$x = \frac{D_x}{D}$$

$$y = \frac{D_y}{D}$$

Never divide by D when $D = 0$. The classification must be determined first.

Verification by Substitution

For a unique solution, substitute the calculated x and y back into both original equations.

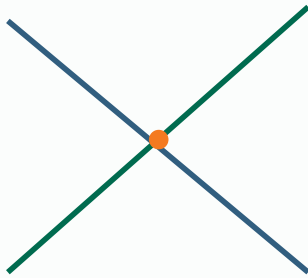
$$L_1 = a_1 x + b_1 y$$

$$L_2 = a_2 x + b_2 y$$

Then compare L_1 with c_1 and L_2 with c_2 . For the required test sets, use values designed to verify cleanly.

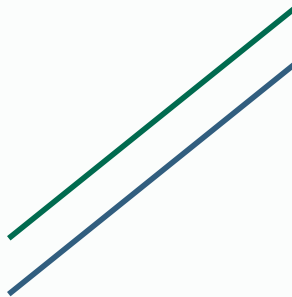
Geometric Interpretation

ONE UNIQUE SOLUTION



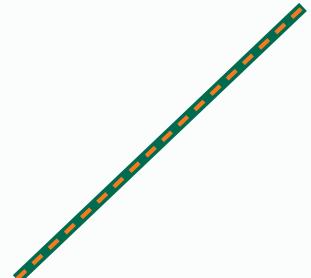
Intersecting lines

NO SOLUTION



Parallel distinct lines

INFINITE SOLUTIONS



Coincident / same line

Worked Reference Case

Consider:

$$2x + y = 8$$

$$x - y = 1$$

The coefficients are $a_1 = 2$, $b_1 = 1$, $c_1 = 8$, $a_2 = 1$, $b_2 = -1$, $c_2 = 1$.

$$D = (2)(-1) - (1)(1) = -3$$

Since $D \neq 0$, there is one unique solution. The system is consistent and independent.

Required Final Report

✓ Display both entered equations.

✓ Display D , D_x , and D_y .

✓ Display the system classification.

✓ State the number of solutions.

✓ State the geometric interpretation.

✓ For a unique solution, display x and y and the intersection point.

✓ For a unique solution, verify both equations by substitution.

✓ For invalid equation input, clearly report the problem.

TWO-VARIABLE LINEAR SYSTEM ANALYZER

EQUATION 1

$$2.00x + 1.00y = 8.00$$

EQUATION 2

$$1.00x - 1.00y = 1.00$$

DETERMINANT ANALYSIS

$$D = -3.00$$

$$D_x = -9.00$$

$$D_y = -6.00$$

SYSTEM CLASSIFICATION

Classification: CONSISTENT AND INDEPENDENT

Number of Solutions: ONE UNIQUE SOLUTION

Geometric Interpretation: The lines intersect.

SOLUTION

$$x = 3.00$$

$$y = 2.00$$

Intersection Point: (3.00, 2.00)

VERIFICATION

$$\text{Equation 1: } 2.00(3.00) + 1.00(2.00) = 8.00 \quad \text{PASSED}$$

$$\text{Equation 2: } 1.00(3.00) - 1.00(2.00) = 1.00 \quad \text{PASSED}$$

Required Test Scenarios

Unique solution: $D \neq 0$.

No solution: $D = 0$ with D_x or D_y non-zero.

Infinitely many solutions: $D = D_x = D_y = 0$.

Negative coefficients: Include at least one negative coefficient.

Decimal coefficients: Include a valid system containing decimal values.

Invalid equation: Make $a_1 = 0$ and $b_1 = 0$, or the same for Equation 2.

Implementation Tips

- Validate both equations before computing the determinants.
- Calculate D , D_x , and D_y once and store them.
- Classify the system before attempting division.
- Only calculate x and y when D is non-zero.
- Keep the classification text and geometric interpretation consistent with the determinant results.
- For the required exact-zero determinant tests, use carefully selected values that produce clean results.

QUALITY ASSURANCE

Testing and Verification Requirements

Each challenge contains multiple logical paths. A single successful run is not sufficient evidence that your program is correct.

What to Test

- Normal valid input
- Each major classification or decision branch
- Lower and upper boundaries
- Invalid input
- Values immediately above or below important thresholds
- Mathematical cases that change the nature of the result

What to Check

- Does the condition match the written rule?
- Are comparisons inclusive or exclusive as intended?
- Is invalid input prevented from reaching later calculations?
- Can any branch be skipped accidentally?
- Does the report explain the result accurately?
- Can you manually verify at least one case?

Boundary testing matters. Conditions such as > 40 and ≥ 40 represent different rules. Your test cases should deliberately include exact boundary values.

PROGRAM PREPARATION

Completion Checklist

- ✓ All three challenges are complete.
- ✓ Each program uses clear variable names.
- ✓ Inputs are validated before dependent calculations.
- ✓ All formulas are implemented according to the challenge specification.
- ✓ Decision logic matches the supplied rules and tables.

- ✓ Outputs are labeled and easy to read.
- ✓ Required test scenarios have been executed.
- ✓ No required branch produces an unexplained result.
- ✓ You can explain the purpose of every major condition in your code.
- ✓ You can modify a threshold or input and predict how the output will change.

Suggested filenames: `CodingChallenge03_LoanDecisionEngine.py`,
`CodingChallenge03_QuadraticAnalyzer.py`, and `CodingChallenge03_LinearSystemAnalyzer.py`.

Final reminder: These challenges assess both programming and reasoning. Your source code should show that you understand the relationship among the input data, mathematical formulas, Boolean conditions, control structures, and final decision.